



Facultad de Ingeniería
Departamento de Ingeniería en Computación e
Informática

Divide y Vencerás

Ivan Mansilla (ivnmansi@umag.cl)

Diego Peralta (dperalta@umag.cl)

Catalina Viñas (caastudi@umag.cl)

Ingeniería en Computación e Informática

Docente: MSc. Jacqueline Aldridge

Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de una expansión a un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV, utilizando algoritmos de ordenamiento y búsqueda. A este se implementaron algoritmos que utilizan el paradigma de divide y vencerás, tal como Merge-Sort, QuickSort, etc. Con esto hecho, se analizó experimentalmente el rendimiento de cada algoritmo, comparando sus tiempos de ejecución y creando gráficos para visualizar tanto sus diferencias como sus complejidades temporales.

ÍNDICE GENERAL

Índice de figuras	v
1. Objetivos	1
1.1. Objetivo Principal	1
1.2. Objetivos Secundarios	1
2. Marco Teórico	2
2.1. Algoritmos de Ordenamiento	2
2.1.1. Merge Sort	2
2.1.2. Merge Sort Optimizado	2
2.1.3. Quick sort	2
2.1.3.1. Pivote último elemento	3
2.1.3.2. Pivote primer elemento	3
2.1.3.3. Pivote elemento aleatorio	3
2.2. Algoritmos de Búsqueda y Selección	3
2.2.1. Búsqueda Binaria	3
2.2.2. Búsqueda por Interpolación	3
2.2.3. Quick Select	4
3. Desarrollo	5
3.1. Algoritmos de Ordenamiento	5
3.1.1. Merge Sort	5
3.1.2. Merge Sort Optimizado	6
3.1.3. Quick sort	7
3.1.3.1. Partición de Lomuto	7
3.1.3.2. Variantes de selección de pivote	7
3.1.3.3. Variante de último elemento	7
3.1.3.4. Variante de primer elemento	7
3.1.3.5. Variante de elemento aleatorio	7
3.1.3.6. Variante de mediana de tres	8
3.1.3.7. Estructura recursiva	8
3.2. Algoritmos de Búsqueda y Selección	8
3.2.1. Búsqueda Binaria Recursiva	8

3.2.2.	Búsqueda por Interpolación	9
3.2.3.	Quick Select	11
3.3.	Aplicación práctica	12
3.3.1.	Ranking de los mejores N deportistas	12

ÍNDICE DE FIGURAS

OBJETIVOS

1.1. Objetivo Principal

Expandir un sistema de ordenamiento y búsqueda de deportistas, agregando algoritmos que utilizan la estrategia de divide y vencerás, con el fin de mejorar la eficiencia del sistema.

1.2. Objetivos Secundarios

- Implementar correctamente los algoritmos de ordenamiento y sus versiones optimizadas según corresponda.
- Implementar correctamente los algoritmos de búsqueda.
- Implementar correctamente el algoritmo de selección (QuickSelect).
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados.
- Evaluar y comprender la complejidad teórica de los algoritmos de ordenamiento, selección y búsqueda.
- Documentar de manera adecuada el código desarrollado y los resultados obtenidos en el análisis experimental.

MARCO TEÓRICO

En este capítulo se describen los conceptos y algoritmos utilizados en el proyecto, centrados en la estrategia de diseño de algoritmos conocida como "Divide y Vencerás".

2.1. Algoritmos de Ordenamiento

2.1.1. Merge Sort

Es un algoritmo basado en la técnica de dividir el problema en subproblemas más pequeños. El proceso consiste en dividir el arreglo original en dos mitades, ordenarlas por separado de manera recursiva y finalmente combinarlas mediante una operación de mezcla (merge) que asegura el orden de los elementos.

Este algoritmo tiene una complejidad temporal de $O(n \log n)$ en todos los casos (mejor, promedio y peor), lo que lo hace eficiente y predecible para grandes conjuntos de datos. Esto se debe a su estructura regular de división recursiva, que no depende del estado inicial de los datos. Además, es un algoritmo estable, aunque su implementación estándar utiliza espacio adicional para la mezcla.

2.1.2. Merge Sort Optimizado

El rendimiento del Merge Sort estándar puede mejorar mediante ajustes prácticos. Una optimización común consiste en establecer un umbral; cuando el tamaño del arreglo es pequeño, el algoritmo cambia a *Insertion Sort* para reducir la carga de la recursividad. También es posible verificar si las dos mitades ya están ordenadas antes de intentar mezclarlas, evitando operaciones innecesarias.

La complejidad asintótica sigue siendo $O(n \log n)$ en todos los casos. Sin embargo, las optimizaciones prácticas (como verificar si las mitades ya están ordenadas) reducen las constantes multiplicativas y mejoran el rendimiento en la práctica, aunque no cambian la complejidad asintótica del algoritmo.

2.1.3. Quick sort

Al igual que el Merge Sort, el Quick Sort es un algoritmo de ordenamiento basado en la técnica de divide y vencerás. Este algoritmo funciona seleccionando un pivote y realizando una partición del arreglo en dos subarreglos: uno con los elementos menores que el pivote y

otro con los elementos mayores. Posteriormente, el mismo procedimiento se aplica de forma recursiva a cada subarreglo hasta que todos los elementos quedan ordenados.

2.1.3.1. Pivote último elemento

En esta variante, el pivote se selecciona como el último elemento del arreglo. Aunque es fácil de implementar, esta estrategia puede llevar a un rendimiento deficiente (complejidad $O(n^2)$) en casos donde los datos ya están ordenados o casi ordenados, ya que el pivote no divide el arreglo de manera equilibrada.

2.1.3.2. Pivote primer elemento

En esta variante, el pivote se selecciona como el primer elemento del arreglo. Similar a la selección del último elemento, esta estrategia también puede resultar en un rendimiento deficiente (complejidad $O(n^2)$) para datos ordenados o casi ordenados, debido a la falta de equilibrio en la partición.

2.1.3.3. Pivote elemento aleatorio

En esta variante, el pivote se selecciona de manera aleatoria dentro del arreglo. Esta estrategia mejora significativamente el rendimiento promedio del Quick Sort, ya que reduce la probabilidad de seleccionar pivotes que conduzcan a particiones desequilibradas. La complejidad promedio y mejor caso sigue siendo $O(n \log n)$, mientras que el peor caso se reduce a $O(n \log n)$ con alta probabilidad, lo que hace que esta variante sea más robusta frente a diferentes tipos de datos.

2.2. Algoritmos de Búsqueda y Selección

2.2.1. Búsqueda Binaria

Es un algoritmo de búsqueda cuyo funcionamiento se basa en dividir repetidamente el arreglo en dos mitades, descartando en cada paso la mitad en la que el elemento buscado sea imposible de encontrar. Para funcionar necesita que los datos estén previamente ordenados.

2.2.2. Búsqueda por Interpolación

Es una mejora sobre la búsqueda binaria para casos donde los datos están distribuidos de manera uniforme. En lugar de buscar siempre en el centro, el algoritmo calcula una posición estimada basándose en el valor buscado y los valores en los extremos del arreglo.

2.2.3. Quick Select

Este algoritmo se utiliza para encontrar el k -ésimo elemento más pequeño en una lista desordenada. Utiliza una lógica similar a Quick Sort (selección de un pivote), pero a diferencia de este, solo continúa la búsqueda en la partición que contiene el índice deseado, lo que reduce significativamente el tiempo de ejecución.

DESARROLLO

En esta sección se detalla la implementación de los algoritmos y las decisiones tomadas para mejorar su desempeño.

3.1. Algoritmos de Ordenamiento

3.1.1. Merge Sort

La implementación de Merge Sort se realizó mediante una estrategia recursiva de tipo *divide and conquer*. El algoritmo divide sucesivamente el arreglo de deportistas en dos mitades hasta alcanzar el caso base, correspondiente a subarreglos de un único elemento, los cuales ya se consideran ordenados.

La función recursiva calcula el punto medio utilizando la expresión:

$$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$$

evitando posibles errores cuando los índices son muy grandes que podrían ocurrir con la expresión clásica:

$$(\text{left} + \text{right}) / 2$$

Una vez ordenadas ambas mitades, se utiliza una función auxiliar `merge` encargada de fusionarlas en un único segmento ordenado. Para ello, se reservan dinámicamente dos arreglos temporales (L y R) que almacenan copias de las sublistas izquierda y derecha respectivamente.

Durante la mezcla, los elementos son comparados mediante la función `compare_by_criteria`, permitiendo ordenar según distintos criterios del sistema (ID, puntaje o cantidad de competencias). Además, la función `should_precede` abstrae la lógica de comparación para soportar tanto orden ascendente como descendente.

El proceso de mezcla recorre simultáneamente ambos arreglos temporales e inserta en el arreglo original el elemento que debe preceder al otro según el criterio seleccionado. Cuando una de las mitades se agota, los elementos restantes de la otra mitad son copiados directamente.

Debido al uso de memoria auxiliar para las sublistas temporales, el algoritmo requiere una complejidad espacial adicional de:

$$O(n)$$

mientras que su complejidad temporal se mantiene estable en:

$$O(n \log n)$$

tanto en el caso promedio como en el peor caso.

3.1.2. Merge Sort Optimizado

La versión optimizada de Merge Sort conserva la misma estructura recursiva y el mismo procedimiento de mezcla que la implementación estándar, pero incorpora una optimización híbrida basada en Insertion Sort para mejorar el rendimiento práctico sobre subarreglos pequeños.

Durante la recursión, antes de seguir dividiendo el arreglo, se evalúa el tamaño del sub-arreglo actual mediante la condición:

```
right - left + 1 <= MERGE_SORT_THRESHOLD
```

Si el tamaño es menor o igual al umbral definido, el algoritmo deja de subdividir y utiliza **Insertion Sort** sobre ese segmento.

Esta optimización se basa en que Insertion Sort posee un costo constante muy bajo y un excelente desempeño en arreglos pequeños o parcialmente ordenados, evitando el overhead asociado a múltiples llamadas recursivas y asignaciones dinámicas de memoria.

La implementación de **Insertion Sort** desplaza los elementos mayores hacia la derecha hasta encontrar la posición correcta del elemento actual (**key**). Al igual que en Merge Sort, las comparaciones se realizan mediante la función **should_precede**, manteniendo compatibilidad con múltiples criterios y órdenes de ordenamiento.

La operación de mezcla sigue utilizando arreglos auxiliares dinámicos para fusionar las mitades ya ordenadas. Además, la implementación incluye validaciones de memoria después de cada llamada a **malloc**. En caso de error de asignación, se liberan inmediatamente los recursos reservados mediante **free**, evitando fugas de memoria.

Aunque la complejidad asintótica permanece en:

$$O(n \log n)$$

la reducción de llamadas recursivas y operaciones de memoria sobre subarreglos pequeños produce una mejora significativa en el rendimiento real del algoritmo.

3.1.3. Quick sort

3.1.3.1. Partición de Lomuto

La partición es la operación central del algoritmo. Recibe el subarreglo delimitado por low y high, tomando siempre el último elemento como pivote. Mantiene un índice i que comienza antes del inicio del subarreglo y marca el límite de los elementos que ya se sabe que deben ir a la izquierda del pivote. El índice j recorre todos los elementos del subarreglo excepto el pivote. Cada vez que encuentra un elemento que pertenece a la izquierda, avanza i e intercambia ese elemento con deportistas[j]. Al terminar el recorrido, coloca el pivote en i + 1, que es su posición definitiva, y retorna ese índice.

3.1.3.2. Variantes de selección de pivote

Todas las variantes comparten exactamente el mismo código de partición y recursión. La única diferencia entre ellas es qué hacen antes de partir, y todas terminan dejando el pivote elegido en la última posición para que Lomuto pueda operar siempre de la misma manera.

3.1.3.3. Variante de último elemento

La variante de último elemento no realiza ninguna acción previa, ya que el pivote ya está donde Lomuto lo espera. Es la más simple pero la más vulnerable: en arreglos ya ordenados el pivote siempre cae en un extremo, lo que degenera el rendimiento a

$$O(n^2)$$

.

3.1.3.4. Variante de primer elemento

La variante de primer elemento simplemente intercambia el primer elemento con el último antes de partir. Resuelve el caso del arreglo invertido pero sigue siendo vulnerable al arreglo ya ordenado, que es el peor caso contrario.

3.1.3.5. Variante de elemento aleatorio

La variante de elemento aleatorio elige un índice al azar dentro del rango y lo mueve al final. Al no depender de la disposición actual del arreglo, rompe los patrones adversos que afectan a los pivotes fijos y garantiza un buen rendimiento promedio sin importar el orden inicial de los datos.

3.1.3.6. Variante de mediana de tres

La variante de mediana de tres examina el primer, el del medio y el último elemento, los ordena entre sí mediante comparaciones directas, y usa la mediana como pivote moviéndola al final. Es la variante más robusta: evita los peores casos de los pivotes fijos, produce particiones más balanceadas que el pivote aleatorio en la mayoría de los casos, y no depende de un generador de números aleatorios.

3.1.3.7. Estructura recursiva

Cada función sigue la misma estructura: si $low < high$, se posiciona el pivote, se particiona el subarreglo obteniendo el índice final del pivote pi , y se llama recursivamente sobre los intervalos $[low, pi - 1]$ y $[pi + 1, high]$. El caso base es implícito: cuando $low \geq high$, el subarreglo tiene un solo elemento o está vacío, por lo que ya está ordenado y no hay nada que hacer.

La complejidad en el caso promedio es:

$$O(n \log n)$$

El peor caso es:

$$O(n^2)$$

y ocurre cuando el pivote siempre queda en un extremo, situación que las variantes de pivote aleatorio y mediana de tres mitigan considerablemente.

3.2. Algoritmos de Búsqueda y Selección

3.2.1. Búsqueda Binaria Recursiva

La función recibe cinco parámetros: el arreglo de deportistas, los índices `left` y `right` que delimitan el rango actual de búsqueda, el criterio de búsqueda `criteria` y el ID objetivo. El parámetro `criteria` está presente únicamente por consistencia con el resto de los algoritmos de búsqueda del proyecto, pero no se utiliza internamente ya que esta función siempre busca por ID.

La estructura del algoritmo sigue un esquema `if/else` anidado. El primer `if` evalúa el caso base: si `left` es mayor que `right`, el rango se agotó sin encontrar el elemento y se retorna `-1`. De lo contrario, se calcula el índice del elemento del medio usando la expresión:

$$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$$

Esta expresión es equivalente a:

$$(\text{left} + \text{right}) / 2$$

pero evita un potencial desbordamiento aritmético en arreglos muy grandes.

Con el índice del medio calculado, se compara su ID con el objetivo. Si son iguales, se retorna el índice directamente. Si el ID del medio es mayor que el buscado, el elemento solo puede estar en la mitad izquierda, por lo que se llama recursivamente con:

$$\text{right} = \text{mid} - 1$$

En caso contrario, el elemento está en la mitad derecha y se llama con:

$$\text{left} = \text{mid} + 1$$

Cada llamada recursiva reduce el rango a la mitad, lo que produce una complejidad temporal de:

$$O(\log n)$$

A diferencia de la versión iterativa, esta versión consume espacio adicional en el stack proporcional a:

$$O(\log n)$$

debido a las llamadas apiladas, aunque en este trabajo de laboratorio no representa un problema.

3.2.2. Búsqueda por Interpolación

La búsqueda por interpolación se implementó como una alternativa optimizada a la búsqueda binaria para arreglos numéricos ordenados y distribuidos de manera relativamente uniforme.

A diferencia de la búsqueda binaria, que siempre inspecciona el elemento central del rango actual, este algoritmo estima una posición probable del elemento objetivo utilizando interpolación lineal. La posición se calcula mediante la expresión:

La posición estimada pos se calcula mediante la fórmula de interpolación lineal, la cual mapea el valor buscado dentro del rango de valores conocidos del arreglo:

$$pos = low + \left[\frac{high - low}{highValue - lowValue} \cdot (targetValue - lowValue) \right] \quad (3.1)$$

Donde:

- **low / high:** Son los índices de los límites actuales del subarreglo.
- **lowValue / highValue:** Son los valores almacenados en dichos índices.
- **targetValue:** Es el valor numérico que se desea encontrar.

La implementación soporta múltiples criterios de búsqueda (ID, **puntaje** y **competencias**) mediante condicionales que extraen dinámicamente el valor numérico correspondiente desde cada deportista.

Una vez estimada la posición, el algoritmo compara el valor encontrado con el objetivo:

- Si ambos coinciden, se retorna el índice encontrado.
- Si el valor es menor que el objetivo, la búsqueda continúa en la mitad derecha.
- En caso contrario, continúa en la mitad izquierda.

La implementación incluye varias validaciones para garantizar robustez:

- Verificación de punteros nulos y arreglos vacíos.
- Control de división por cero cuando los valores extremos son iguales.
- Comprobación de que el valor objetivo se encuentre dentro del rango actual.
- Validación de que la posición interpolada permanezca dentro de los límites válidos del arreglo.

Cuando los datos poseen una distribución uniforme, la complejidad promedio puede aproximarse a:

$$O(\log \log n)$$

superando a la búsqueda binaria tradicional. Sin embargo, en distribuciones desbalanceadas o no uniformes, el rendimiento puede degradarse hasta:

$$O(n)$$

3.2.3. Quick Select

Para obtener el elemento ubicado en una posición específica sin ordenar completamente el arreglo, se implementó el algoritmo *Quick Select*. Este algoritmo reutiliza la lógica de partición de Quick Sort, pero únicamente continúa la búsqueda sobre la partición que contiene el elemento deseado.

La operación principal del algoritmo es la función `partition`, basada en el esquema de Lomuto. En esta implementación, el pivote corresponde al último elemento del subarreglo. Durante el recorrido, todos los elementos que deben preceder al pivote según el criterio y orden seleccionados son intercambiados hacia la izquierda mediante la función `swap_deportistas`.

La comparación se realiza utilizando:

`compare_by_criteria`

lo que permite seleccionar elementos según distintos atributos del deportista, mientras que el parámetro `order` permite trabajar tanto en orden ascendente como descendente.

Una vez finalizada la partición, el pivote queda ubicado en su posición definitiva dentro del arreglo. A partir de esto, el algoritmo evalúa tres casos:

- Si el índice del pivote coincide con k , el elemento buscado fue encontrado.
- Si k es menor que el índice del pivote, la búsqueda continúa únicamente en la partición izquierda.
- Si k es mayor, la búsqueda continúa únicamente en la partición derecha.

A diferencia de Quick Sort, Quick Select nunca procesa ambas particiones simultáneamente, lo que reduce considerablemente el trabajo realizado.

El caso base ocurre cuando el subarreglo contiene un único elemento:

`left == right`

momento en el que dicho elemento es retornado directamente.

La complejidad temporal promedio del algoritmo es:

$$O(n)$$

ya que en cada iteración solo se continúa explorando una partición del arreglo. Sin embargo, al igual que Quick Sort, el peor caso ocurre cuando las particiones quedan extremadamente desbalanceadas, alcanzando una complejidad de:

$$O(n^2)$$

aunque este comportamiento es poco frecuente en datos aleatorios.

3.3. Aplicación práctica

3.3.1. Ranking de los mejores N deportistas

Para obtener el ranking de los N mejores deportistas según puntaje, se implementó la función `get_top_n_athletes`, que recibe el arreglo de deportistas, su cantidad total y el valor N . La función reserva un arreglo de N posiciones y lo completa llamando a `get_kth_athlete` con $k = 1, 2, \dots, N$, obteniendo cada puesto de forma independiente mediante Quick Select.

A diferencia de ordenar el arreglo completo, Quick Select localiza el k -ésimo elemento sin necesidad de ordenar todos los datos. Su costo promedio es:

$$O(n)$$

por llamada, lo que resulta en un costo total de:

$$O(N \cdot n)$$

para el ranking completo, frente al costo:

$$O(n \log n)$$

de un ordenamiento completo. Esto lo hace más eficiente cuando N es pequeño respecto al total de deportistas.

La función `show_ranking` se encarga de cargar los datos desde el archivo CSV, invocar `get_top_n_athletes` para obtener los resultados y mostrarlos utilizando las funciones de formato ya existentes en el sistema.